

PMUフェーザー推定アルゴリズム解説

最小二乗法（LS）と DFT 補完

対象規格: IEEE C37.118.1 (Annex C 参照アルゴリズム)

実装対象: Arduino Mega 2560 + ZMPT101B

目次

- 共通の前提：観測信号モデル
- 最小二乗法（Least Squares）
- DFT補完（Discrete Fourier Transform）
- TVE（全ベクトル誤差）
- 2手法の比較
- 実装上の注意点

1. 共通の前提：観測信号モデル

電力系統の基本波信号は次式で表される。

$$x(t) = X_m \cdot \cos(\omega_0 t + \phi) + \varepsilon(t)$$

X_m : 振幅（ピーク値）

ω_0 : 角周波数 = $2\pi \cdot f_0$

ϕ : 初期位相

ε : ノイズ・高調波など

フェーザー表現（IEEE C37.118.1 定義）：

$$\begin{aligned} X &= (X_m / \sqrt{2}) \cdot e^{j\phi} \\ &= X_r + j \cdot X_i \end{aligned}$$

ここで $X_r = (X_m / \sqrt{2}) \cdot \cos(\phi)$ 、 $X_i = (X_m / \sqrt{2}) \cdot \sin(\phi)$ が実部・虚部。

ADC でサンプリングした離散信号 $x[k] = x(k \cdot T_s)$ から X_m と ϕ を推定するのが目標。

2. 最小二乗法（Least Squares）

2.1 基本アイデア

$\cos(\omega_0 t)$ と $\sin(\omega_0 t)$ の線形結合として信号をモデル化し、サンプル点への当てはまりが最もよい係数 A 、 B を求める。

$$x[k] = A \cdot \cos(\omega_0 \cdot k \cdot T_s) + B \cdot \sin(\omega_0 \cdot k \cdot T_s) \quad (k = 0, 1, \dots, N-1)$$

A と B が分かれば：

$$\begin{aligned} \text{振幅} \quad X_m &= \sqrt{A^2 + B^2} \\ \text{位相} \quad \phi &= \text{atan2}(-B, A) \quad [\text{rad}] \end{aligned}$$

2.2 行列形式

N 個の方程式を行列で書くと：

$$\mathbf{x} = \mathbf{H} \cdot \boldsymbol{\theta} + \boldsymbol{\varepsilon}$$

$$\mathbf{x} = [x[0], x[1], \dots, x[N-1]]^T \quad (N \times 1 \text{ 観測ベクトル})$$

$$\mathbf{H} = \begin{bmatrix} \cos(\omega_0 \cdot 0 \cdot T_s) & \sin(\omega_0 \cdot 0 \cdot T_s) \\ \cos(\omega_0 \cdot 1 \cdot T_s) & \sin(\omega_0 \cdot 1 \cdot T_s) \\ \vdots & \vdots \\ \cos(\omega_0 \cdot (N-1) \cdot T_s) & \sin(\omega_0 \cdot (N-1) \cdot T_s) \end{bmatrix} \quad (N \times 2 \text{ 観測行列})$$

$$\boldsymbol{\theta} = [A, B]^T \quad (2 \times 1 \text{ パラメータベクトル})$$

$N > 2$ のとき **過決定系**（方程式の数 > 未知数）になる。

2.3 最小二乗解の導出

残差の二乗和 $J(\boldsymbol{\theta}) = \|\mathbf{x} - \mathbf{H}\boldsymbol{\theta}\|^2$ を最小化する $\boldsymbol{\theta}$ を求める。

$$\partial J / \partial \boldsymbol{\theta} = 0 \rightarrow \mathbf{H}^T \mathbf{H} \cdot \hat{\boldsymbol{\theta}} = \mathbf{H}^T \mathbf{x}$$

これを **正規方程式** と呼ぶ。解は：

$$\hat{\boldsymbol{\theta}} = (\mathbf{H}^T \mathbf{H})^{-1} \cdot \mathbf{H}^T \cdot \mathbf{x} = \mathbf{H}^+ \cdot \mathbf{x}$$

$\mathbf{H}^+ = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T$ を **Moore-Penrose 疑似逆行列** という。

2.4 2×2 行列の具体的な計算

$\mathbf{H}^T \mathbf{H}$ は 2×2 対称行列なので逆行列を解析的に求められる：

$$H^T H = \begin{bmatrix} \sum \cos^2(\omega_0 k T_s) & \sum \cos(\omega_0 k T_s) \sin(\omega_0 k T_s) \\ \sum \cos(\omega_0 k T_s) \sin(\omega_0 k T_s) & \sum \sin^2(\omega_0 k T_s) \end{bmatrix}$$

$$= \begin{bmatrix} c_{00} & c_{01} \\ c_{01} & c_{11} \end{bmatrix}$$

$$\det = c_{00} \cdot c_{11} - c_{01}^2$$

$$A = (c_{11} \cdot (H^T x)_0 - c_{01} \cdot (H^T x)_1) / \det$$

$$B = (-c_{01} \cdot (H^T x)_0 + c_{00} \cdot (H^T x)_1) / \det$$

$$(H^T x)_0 = \sum \cos(\omega_0 k T_s) \cdot x[k], \quad (H^T x)_1 = \sum \sin(\omega_0 k T_s) \cdot x[k]$$

2.5 Arduino 実装の要点

```
c

// double精度で累積 → 桁落ち防止
double HtH00=0, HtH01=0, HtH11=0;
double Htx0=0, Htx1=0;
double omega = 2.0 * M_PI * f0 * Ts;

for (uint16_t k = 0; k < N; k++) {
    double c = cos(omega * k);
    double s = sin(omega * k);
    HtH00 += c * c;    HtH01 += c * s;    HtH11 += s * s;
    Htx0  += c * x[k]; Htx1  += s * x[k];
}

double det = HtH00 * HtH11 - HtH01 * HtH01;
double A = ( HtH11*Htx0 - HtH01*Htx1) / det;
double B = (-HtH01*Htx0 + HtH00*Htx1) / det;
```

計算量は **O(N)**（前計算の H^+ を使い回す場合は乗算 **2N** 回のみ）。

2.6 特徴まとめ

観点	内容
推定精度	最小分散不偏推定量 （ガウスノイズ下で最良線形不偏推定）
周波数追従	f_0 を更新するだけで H を再構成でき、離調に強い
高調波除去	H に高調波列を追加すれば同時推定・除去が可能
計算量	O(N) — FFT 不要
RAM使用量	少ない（行列格納が不要）

観点	内容
制約	f_0 の初期値が必要（ゼロクロス法などで事前推定）

3. DFT補完（Discrete Fourier Transform）

3.1 基本アイデア

N 点の離散フーリエ変換を計算し、基本波周波数に対応する **bin** の複素振幅からフェーザーを読み取る。

$$\begin{aligned} X[k] &= \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N} \\ &= \sum x[n] \cdot \cos(2\pi kn/N) - j \cdot \sum x[n] \cdot \sin(2\pi kn/N) \\ &= \text{Re}[k] + j \cdot \text{Im}[k] \end{aligned}$$

3.2 基本波 bin の選択

サンプリング周波数 F_s 、サンプル数 N のとき、周波数分解能は $\Delta f = F_s/N$ 。

$$\text{基本波 bin: } k_0 = \text{round}(f_0 / \Delta f) = \text{round}(f_0 \cdot N / F_s)$$

例: $F_s=2000\text{Hz}$, $N=200$, $f_0=50\text{Hz}$ $\rightarrow$ $k_0 = \text{round}(50 \times 200 / 2000) = 5$

3.3 フェーザー変換

bin k_0 の DFT 値からフェーザーを求める：

$$\begin{aligned} \text{振幅 } X_m &= 2 \cdot |X[k_0]| / N = 2 \cdot \sqrt{\text{Re}[k_0]^2 + \text{Im}[k_0]^2} / N \\ \text{位相 } \phi &= \text{angle}(X[k_0]) = \text{atan2}(\text{Im}[k_0], \text{Re}[k_0]) \end{aligned}$$

係数 $2/N$ は片側スペクトル補正（両側対称なので基本波成分を2倍）。

3.4 高調波補完

DFT の最大の強みは全周波数成分を一括計算できる点。

2次・3次高調波のフェーザーは $k = 2k_0$ 、 $3k_0$ の bin を読むだけ。

$$\begin{aligned} \text{2次高調波: } k_2 &= 2 \cdot k_0 \rightarrow X[k_2] \rightarrow X_{m_2}, \phi_2 \\ \text{3次高調波: } k_3 &= 3 \cdot k_0 \rightarrow X[k_3] \rightarrow X_{m_3}, \phi_3 \end{aligned}$$

$$\text{全高調波歪率 THD} = \sqrt{X_{m_2}^2 + X_{m_3}^2 + \dots} / X_{m_1} \times 100 \text{ [\%]}$$

3.5 Arduino 実装の要点

```
c

int bin = (int)roundf(targetFreq / (Fs / N));

double re = 0, im = 0;
double twoPiK_N = 2.0 * M_PI * bin / N;

for (uint16_t n = 0; n < N; n++) {
    re += x[n] * cos(twoPiK_N * n);
    im -= x[n] * sin(twoPiK_N * n);    // DFT の虚部は -sin
}

float amp    = (float)(2.0 * sqrt(re*re + im*im) / N);
float phase = (float)(atan2(im, re) * 180.0 / M_PI);
```

計算量は $O(N^2)$ (素朴DFT)。FFT を使えば $O(N \log N)$ に削減可能。

注意: サンプル数 N が整数サイクルでない場合 ($f_0 \times N/Fs$ が整数でない場合) は
スペクトル漏れ (spectral leakage) が発生し、振幅・位相の誤差が増大する。
この場合はウィンドウ関数 (Hanning, Hamming など) で漏れを抑制する。

3.6 ウィンドウ関数の適用

```
c

// Hanning ウィンドウの例
float w = 0.5f * (1.0f - cosf(2.0f * M_PI * n / (N - 1)));
re += x[n] * w * cos(twoPiK_N * n);
im -= x[n] * w * sin(twoPiK_N * n);

// ウィンドウ補正係数 (振幅を回復)
float coherentGain = 0.5f;    // Hanning の場合
amp = (float)(2.0 * sqrt(re*re + im*im) / N / coherentGain);
```

3.7 特徴まとめ

観点	内容
推定精度	整数サイクル条件を満たせば高精度
周波数追従	整数サイクル前提のため離調で誤差増大 (漏れ対策が必要)
高調波解析	全周波数成分を一括取得できる (最大の強み)
計算量	$O(N^2)$ (素朴) / $O(N \log N)$ (FFT)

観点	内容
実装の容易さ	ライブラリが豊富（FFTW, arduinoFFT など）
漏れ対策	ウィンドウ関数が必要（M クラス要件）

4. TVE（全ベクトル誤差）

4.1 定義（IEEE C37.118.1）

LS推定をリファレンス、DFT推定を被評価値とするとき：

$$\text{TVE} = \frac{\sqrt{(\text{Xr_dft} - \text{Xr_ls})^2 + (\text{Xi_dft} - \text{Xi_ls})^2}}{\sqrt{\text{Xr_ls}^2 + \text{Xi_ls}^2}} \times 100 \quad [\%]$$

ここで $\text{Xr} = (\text{Xm}/\sqrt{2}) \cdot \cos(\phi)$ 、 $\text{Xi} = (\text{Xm}/\sqrt{2}) \cdot \sin(\phi)$ は RMS フェーザーの実部・虚部。

4.2 合否基準

クラス	最大 TVE	用途
P クラス（Protection）	1%	最小遅延・保護継電器
M クラス（Measurement）	1%	高精度計測

PMU が IEEE C37.118.1 に適合するには、定常状態で $\text{TVE} \leq 1\%$ を満たす必要がある。

4.3 TVE が大きくなる原因

原因	対策
周波数離調（ $f \neq f_0$ ）	周波数追従 LS または Hanning ウィンドウ
高調波混入	LS に高調波項追加 / M クラス用フィルタ
ADC ノイズ	平均化・増やす N
時刻誤差	GPS 同期（本実装では未実装）

5.2手法の比較

5.1 特性比較表

項目	最小二乗法（LS）	DFT
IEEE C37.118.1	Annex C 参照アルゴリズム	Annex C 参照アルゴリズム
推定対象	基本波のみ（拡張で高調波も可）	全周波数成分
周波数離調への強さ	◎ H再構成で対応	△ 漏れが発生
高調波除去	○ H に追加項で対応	◎ bin を読むだけ
計算量（実行時）	◎ O(N)	○ O(N ²) / FFT は O(N log N)
RAM 使用量	◎ 小（行列格納不要）	○ 中
ノイズ耐性	◎ 最小分散推定	○ ウィンドウ関数依存
実装の容易さ	○ 行列計算が必要	◎ ライブラリ豊富
Arduino Mega 適性	◎	○（素朴 DFT は重い）

5.2 用途別の使い分け

基本波フェーザー精度を最優先

→ 最小二乗法（LS）

高調波解析・THD計測

→ DFT（bin を複数読む）

系統周波数が不安定

→ LS（周波数追従が容易）

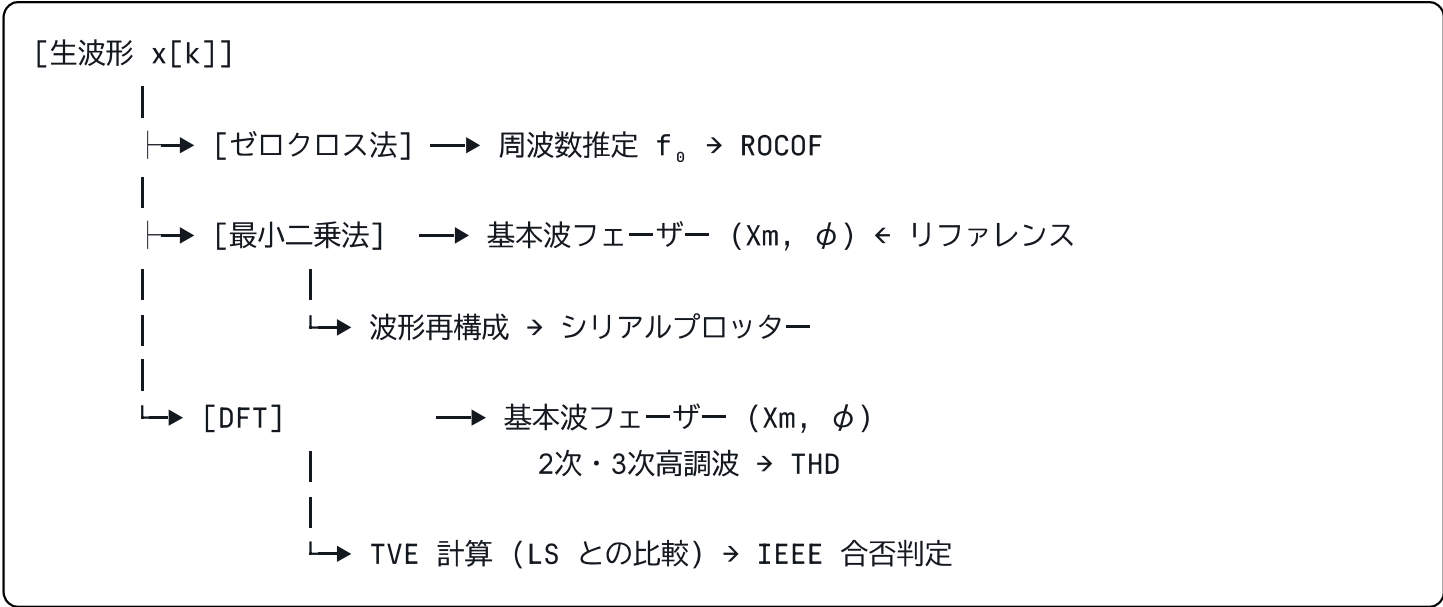
整数サイクルが保証される場合

→ DFT（実装簡単・高速）

IEEE C37.118.1 準拠の厳密評価

→ LS+DFT 両方計算して TVE 比較

5.3 本実装の構成



6. 実装上の注意点

6.1 サンプリング設計

50Hz 系統での推奨設定:

F_s = 2000 Hz (1 サイクルあたり 40 サンプル)
 N = 200 (5 サイクル分 → 周波数分解能 10 Hz)
 k_0 (DFT) = 5 (50 / 10 = 5 番目の bin)

整数サイクル条件: $f_0 \times N / F_s = 50 \times 200 / 2000 = 5.0$ ✓ 整数

6.2 数値精度

```
c
// NG: float では桁落ちで精度劣化
float HtH00 += cosf(...) * cosf(...); // 累積誤差が大きい

// OK: double で累積
double HtH00 += cos(...) * cos(...); // Arduino Mega は double=64bit
```

6.3 DCオフセット除去

ZMPT101B は出力を $(V_{cc}/2)$ にバイアスするため、フェーザー推定前に必ず除去する。

```
c
float dcOffset = 0;
for (int i = 0; i < N; i++) dcOffset += x[i];
dcOffset /= N;
for (int i = 0; i < N; i++) ac[i] = x[i] - dcOffset;
// → ac[] を LS・DFT に渡す
```

6.4 キャリブレーション

センサー出力電圧から実電圧への変換係数 (SENSOR_RATIO) は実測値で校正する。

手順:

1. 既知の実効値電圧 (例: 100 Vrms) を入力
2. シリアルモニタの "RMS (sensor)" 値を読む (例: 0.20 V)
3. $\text{SENSOR_RATIO} = 100.0 / 0.20 = 500.0$ に設定

6.5 IEEE C37.118.1 への適合に向けた追加課題

本実装は基本的な計測アルゴリズムを実装しているが、完全な規格適合には以下が必要。

要件	現状	対策
GPS 時刻同期	未実装	u-blox ZED-F9T + PPS 信号
レポートレート規定	未実装	10～50 fps の設定機能
動特性試験（変調・ランプ・ステップ）	未実装	模擬信号発生器でテスト
M クラス出力フィルタ	未実装	低遅延 FIR フィルタ追加
レイテンシ計測	未実装	タイムスタンプとデータ送信時刻の差分

参考文献

- IEEE C37.118.1-2011: *IEEE Standard for Synchrophasor Measurements for Power Systems*
- IEEE C37.118.1a-2014: Amendment (F & ROCOF 要件緩和、M クラスアルゴリズム更新)
- Ken Martin, "Synchrophasor Measurement Standard - IEEE C37.118.1", IEEE JCTM, 2015